

PROG1

Computer Programming 1

Grade 12, Semester 1 study outline

A summary of section headings and key terms, made by students for revision. It is not the handout and does not replace it.

01. Introduction to Computers and Programming

Introduction to Computers and Programming

I. HUMBLE BEGINNINGS

- Today, a computer is a programmable machine which has 2 principal characteristics:

1. Responds to a specific set of instructions in a well-defined manner

2. Executes a prerecorded list of instructions

- Abacus: It is a Latin word derived from the Greek words "abax" or "abakon" which means a sand table for drawing, calculating, etc.
- It is considered as one of the (if not "the") very first devices that were used to do addition and subtraction operations as well as count large numbers and manage ancient inventory of goods.
- Pascaline: It is a device that was invented by the French mathematician-philosopher Blaise Pascal.
- Difference Engine: It is a mechanical calculator conceptualized in 1821 by Charles Babbage who is known as the "Father of Computer/Computing".
- In 1834, an improved Difference Engine was conceptualized and was called Analytical Engine which was developed to cater to more complex operations.
- Unfortunately, these 2 machines were never physically developed during Babbage's time.
- Ada Augusta King-Noel, Countess of Lovelace or simply known as Ada Lovelace: -- 1 of 4 -- She was an English mathematician and writer who continued the work of Babbage on Analytical Engine.
- Tabulating Machine: It was invented by Herman Hollerith, who is a Mining Engineer and also a Mechanical Engineering teacher at MIT.
- In 1911 there was a merger changing the TMC into the Computer Tabulating Recording Company (CTR).
- Harvard Mark I: It is the first computer-like, room-sized machine developed by professor Howard Aiken in 1944 and was jointly sponsored by IBM and Harvard University.
- Navy during World War II and was the first of a series of computers designed by Aiken (followed by Mark II, Mark III, and Mark IV).

II. GENERATIONS OF COMPUTERS

- 1st Generation The first computer systems used vacuum tubes for circuitry and magnetic drums for memory.
- Computers in this generation used machine language (0s and 1s), which is the lowest-level programming language understood by computers and they can only solve one problem at a time.
- Examples of computers invented that time are the ENIAC (Electronic Numerical Integrator and Calculator) and UNIVAC (Universal Automatic Computer).
- 2nd Generation In the late 1950s, the invention of transistors resulted in smaller, faster, more reliable, and more energy- efficient computers.
- There was a transition from a cryptic machine language to assembly languages which allowed programmers to specify instructions in words (not in 0s and 1s).
- 2 early high-level programming languages, FORTRAN (Formula Translator) & COBOL (Common Business-Oriented Language) were introduced during that time.
- 3rd Generation Transistors were replaced by tiny integrated circuits (IC).
- 4th Generation In 1970, the microprocessor, an entire CPU on a single chip, was invented.
- 4th generation devices are based on Very Large-Scale Integration (VLSI).
- VLSI is the process of integrating or embedding hundreds of thousands of transistors on a single silicon semiconductor microchip.
- 5th Generation 5th generation devices are based on Ultra Large-Scale Integration (ULSI) and artificial intelligence.
- ULSI is the process of integrating or embedding millions of transistors on a single silicon semiconductor microchip.

02. Fundamental Programming Concepts

Fundamental Programming Concepts

I. REVIEW: COMPUTER

- The modern-day definition of a computer is that it is an electronic machine that is programmed to both respond to and execute a specific set of commands or instructions.
- Examples include a mouse, keyboard, web camera, scanner, and magnetic strip card reader.
- Storage devices: These are hardware (and now virtual too) devices that are used for storing data.
- These include hard disk drives, SIM cards, Digital Video Discs (DVDs), Dropbox, and Google Drive.
- The Central Processing Unit (CPU) - the processor and not the "system unit" case - is the one in charge of the Processing aspect.
- These devices include a monitor, an LCD projector, a printer, and a speaker.

II. KEY PROGRAMMING TERMINOLOGIES

- Program: -- 1 of 6 -- In its raw form, it is a set of instructions written in code (keywords and symbols) that a computer understands and, when executed, produces a predetermined output.
- It also refers to the finished product or a specific application, such as Microsoft Word, Photoshop, and Skype.
- Programming: It is the process of producing a set of instructions for the computer to execute.
- If a program is a specific kind of dish, programming is the process of cooking that dish.
- It is a process of problem solving, and it is a creative process because there are many ways to solve a problem.
- Also, you can take the usual route or explore some shortcuts. "Computer programming is an art, because it applies accumulated knowledge to the world, because it requires skill and ingenuity, and especially because it produces objects of beauty." (Donald Knuth, American computer scientist, mathematician, and professor...)
- Programmer: He/she is a person who uses a particular programming language to create instructions (programs) for a computer to execute.
- Mobile Apps - As you visit Apple's App Store or Google's Play Store, you will realize that there is an application for everything.
- Games - Games are composed of millions of lines of code.
- Programming Language: It is a formal and recognized system of symbols and rules that allows a programmer to translate an algorithm into codes which the computer can execute, through the help of a programming language software.

1. Low-level Languages - These include the Machine Language and Assembly Language

- Machine language or machine code is the default language of a computer.

LOAD rate

MULT hours

STOR wages

- Common examples are C, C++, C#, and Java.

III. ALGORITHM

- It is a set of instructions (but not yet in pure code form) for solving a specific problem.
- It is a "well-ordered collection of unambiguous and effectively computable operations that, when executed, produces a result and halts in a finite amount of time".
- Well-ordered: Tasks/instructions inside an algorithm must be in the correct order.
- If the order is not right, we may be confused as to which instruction should be done first and which should be performed next.
- Has unambiguous operations: An algorithm must be clear so that it does not need to be simplified.
- It must be told to search for the smallest number, how to find the smallest number, how to compare numbers together, etc.
- Has effectively computable operations: Operations in an algorithm must be doable or valid.
- If an algorithm doesn't produce some result, we can never be certain whether our solution is correct.
- Halts in a finite amount of time: Algorithms should be composed of a finite (limited, exact) number of operations and executed within a specific period of time.
- Every algorithm must reach a point that tells it to stop.
- Although Ada Lovelace is the first person to have written a programmable algorithm, the word "algorithm" was derived

from the name of Abu Ja'far Muhammad Ibn Musa Al-Khwarizmi (780 AD - 850 AD), a Persian polymath.

- Natural Language Construction The best example is a recipe.

1. Prepare an empty glass (Glass C)
2. Pour Glass A content (Coke) into Glass C
3. Pour Glass B content (Sprite) to Glass A
4. Pour the Glass C content (Coke) into Glass B
5. Glass A now contains Sprite, and Glass B contains Coke

START

OUTPUT glassA

OUTPUT glassB

END

- A flowchart, on the other hand, is a better way of communicating the logic of a particular program.

- What Is An Algorithm and Why Are They Important.

Key terms

He - someone who runs a program. Unlike the programmer, he/she doesn't have to understand

Although Ada Lovelace - first person to have written a programmable algorithm, the word "algorithm"

Natural language - not the best way for writing program algorithms, as it is too wordy and can be

03. Program Design Process

Program Design Process

I. ALGORITHMIC MINDSET

- The idea of coding is valuable and wonderful, but the idea everyone should learn to code?
- That is a baseline skill for every professional in the workplace, no matter what the role is." (Martin Chavez, former

Goldman Sach's CFO and CIO)

- It is a soft skill that can help anyone survive and thrive in his/her profession.
- Review: An algorithm must...

Be well-ordered

Have unambiguous operations

Have effectively computable operations

Produce a result

Stop in a finite amount of time

- Whether the program you are creating is simple or complex, the characteristics mentioned above should be applied.

1. Prepare a temporary variable
2. Get input value for 1st variable
3. Get input value for 2nd variable
4. Temporary variable takes the value of the 1st variable
5. 1st variable takes the value of the 2nd variable
6. 2nd variable takes the value of the temporary variable
7. Display the new values of the variables

START

INITIALIZE num1, num2, temp //variables

INPUT num1 //get input for 1st variable

INPUT num2 //get input for 2nd variable

PRINT num1, num2 //display swapped values

STOP

II. PHASES OF PROGRAM DESIGN

- Problem Solving Phase / Program Logic (Algorithm) Formulation Consider this as the planning phase of computer programming.
- The goal of this phase is not to produce an executable program but an algorithm that solves the problem.

- The following are some practical steps in problem-solving: - Identify the Problem - What is the need to be addressed?
- Determine the expected output. -- 2 of 4 -- - Understand the Problem - Think of how the program should flow in order to produce the expected result.
- Tracing is also a skill that a programmer should possess and this will be applied in the succeeding lessons.
- It is an error where, upon tracing, the algorithm runs but fails to produce the desired output (ex. wrong computation, wrong display, etc.).
- It could also mean there is a wrong flow of instructions and some parts are disconnected. b.
- Implementation Phase / Actual Execution The goal of this phase is to produce an executable program.
- This phase includes the following activities: - Coding/Programming - This is the process of writing the actual program.
- Another type of error that can be detected is the run-time error which occurs during the execution of the program.
- Dividing a number by zero and stack overflow (the program attempts to use more space or memory than what is available) are examples of this error. - Debugging and Testing - Debugging is the process of finding errors (refer to the 3 errors mentioned above) and correcting them.
- Testing, on the other hand, is a verification if the completed program or software package functions according to the expectations defined.

1. Programmer's Documentation

2. User's Documentation

- This is a guide for the end-user.
- It consists of instructions for installing the program as well as a manual on how to use the program.
- Nevertheless, whether you are a beginner or an expert, preparing and understanding the program's logic will always be necessary. -- 3 of 4 -- In this Programming 1 course, you will be focusing on the Algorithm Formulation phase, specifically using flowchart. "You might not think that programmers are artists, but pro...

Key terms

Whether the program you - creating is simple or complex, the characteristics mentioned above should be

04. Basic Elements of Programming

Basic Elements of Programming

- Topic: Basic Elements of Programming

BASIC ELEMENTS OF PROGRAMMING

- Regardless of the level of programming language to be used and the kind of program to be created, there are different elements that should be present.

1. Data

- It refers to any value that the computer gets as an input, stores for future use, and transforms into a meaningful output.

Different Types of Data

- A data type is a classification of data telling the compiler/assembler/interpreter how the programmer wants to use the data.
- It also specifies which type of value a variable stores and what type of operations can be applied to it.

Data Type Used For Example

Integer Whole numbers that can

Float

Point)

Decimal numbers or

Character A single character, such

String A text or a sequence of

Hello World,

GFA, hi2021

Boolean Logical values TRUE, FALSE

2. Variable

- It is a specific area/location in the computer memory where a value/data is stored and retrieved.
- There must be a variable that shall receive/hold every value that is inputted into the computer.

- There is a specific container for a soft drink, milk, alcohol, black pepper, powdered juice, etc.
- For example, Pi is always 3.14159... and other values that a programmer will define to remain the same all throughout the program.

3. Input

- This is the process of getting data from the programmer or user to be processed by the computer.
- Basic example of this is capturing some text that the user types on the keyboard.
- 10) doesn't require the user to provide input.

4. Output

- This is the process of producing processed data.
- An example is printing out something on the terminal screen.
- By default, all programs should produce results.

5. Processes/Operations

- What lie between or connect the input and output elements are the processes that are running inside a program.
- These processes include the following: - Assigning Values to Variables and Changing the Values of Variables Empty variables can be assigned values but also, variables with existing values can be overwritten with new values.
- One good example of this is our swapping problem in Lesson 3.
- Then, we have also changed the value of the variable "num1" when it took the value of the variable "num2". - Computing/Calculations As you learned in Lesson 1, the very basic function of a computer is to compute.
- In Lesson 6, you will learn the different types of operators used in programming.

1. The variable "num1" has an initial value of 5

2. "num1" "takes the value of" "num1" (which is 5) plus 5

3. Therefore, the new value of num1 is 10

1. The variable "num1" has an initial value of 5

2. The variable "num2" has an initial value of 10

3. "num2" "takes the value of" "num1" (which is 5) plus 15

- Therefore, the new value of "num2" is 20.
- Another example of the effects of comparing values is directing the program flow.

IF number is positive,

- As an example of the IPO application, think of a program that will convert Fahrenheit temperatures into Celsius temperatures.
- The program must: - Ask the user for the Fahrenheit temperature (input) - Perform a calculation to convert the Fahrenheit temperature into the corresponding Celsius temperature (process) - Display the Celsius temperature (output) Here's a practical example applying the IPO model:

6. Subroutines/Functions

- It is the element of the programming that allows a programmer to reuse a snippet of code in one location which can be used by other parts of the program over and over again.
- If the user wants to know the area of a -- 3 of 4 -- circle, given the value for radius, you will have a code that states the formula is .
- Now, the user also wants to compute the volume of a cylinder and the formula is .
- You can create a function that computes for the area of a circle and just refer to it again when the user wants to compute for the volume of a cylinder then multiply its result to the h (height) rather than typing the again in the cylinder section of the program.
- 5 Basic Elements Of Programming | Information System Assignment Help. (n.d.).

Key terms

- Pi - always 3.14159... and other values that a programmer will define to remain the same all throughout the

05. Logical-Control Structures in Programming

Logical-Control Structures in Programming

CONTROL STRUCTURES

- The decisions based on certain conditions that affect the direction of the program flow are made by the control/logical structures.

- 3 Control/Logical Structures:

1. Sequence Logic / Sequential Flow

- This is the default control structure.
- As the name suggests, it is the execution of code line by line.

2. Selection Logic / Conditional Flow

- This is the structure that enables the program to select or choose between two or more flows.

3. Iteration Logic / Repetitive Flow

- It is a programming structure that allows line(s) of code to be repeated or executed multiple times as needed in the program.
- In C, C++, JavaScript, Python, Java and other high-level programming languages, this structure is facilitated by the 3 major kinds of loop statements: while, do-while, and for loops.
- Here are the sample C/C++ syntaxes of these looping statements: while Loop while (condition) { /*Condition is checked first, then once satisfied, the program performs the statements here and repeats them as long as the condition above is met*/ } do-while Loop do { /*Statements here are executed first, then at the bo...
- If the condition is satisfied, the whole do-while section will repeat as needed*/ } while (condition); for Loop for (initialization; condition; increment) -- 5 of 8 -- { /*Statements here are repeated for as long as the condition above is met*/; } You would notice that the common thing among the 3 loops above is th...
- Just like in the Conditional Flow, it is the condition part of the code that determines the next route to be taken by the program in a Repetitive Flow structure.

Key terms

Control structures - considered as the "building blocks" of computer programs. They specify the flow of
Different statements - performed, depending on which side of the condition is met (true or false)

06. Types of Operators Part 1

Types of Operators Part 1

- Topic: Types of Operators Part 1

OPERATORS

- They are used in assigning, computing, and comparing values.

1. Assignment Operator

- This operator denoted by '=' (equal) assigns/sets a value to an empty variable or changes/resets the existing value of a variable.
- The one on the right of the operator is a value or an expression that produces a value.
- Examples: a. age = 30 - age is the variable, 30 is the value - The statement above can be read as "age is assigned the value of 30" or "age gets/takes the value of 30" b. x = x + 4 - x is the variable that will store the result of the expression on the right of the operator. - x + 4 is an expression to be evaluated,...

1. num1 is assigned the value of 5

2. num2 is assigned the value of 10

3. num3 is assigned the value of the sum of num1 and num2

- - Therefore, num3 = 15 In conjunction with the Arithmetic Operators (which will be discussed next), here are other ways of using the Assignment operator:

Compound Assignment Operators

Symbol Description Example

- -- 1 of 4 -- += Addition Assignment

It adds the right operand to

It subtracts the right operand

C - A

It multiplies the right operand

It divides the left operand

C / A

It divides the left operand

2. Arithmetic Operators

Operator Name Symbol Usage Result

Subtraction - x - y Difference

Division / x / y Quotient

- This type of division is called integer division, and it is what we learned from our earlier Math subjects.
- In the real world of data manipulation, there are things that are always handled in whole numbers (integer data type).
- Another good example of integer division is when applied to living things like puppies.
- It is the other part of the answer in an integer division.
- In earlier Math subjects, you would say, "Eleven divided by four is two remainder three." In many programming languages, the symbol for the modulus operator is the percent sign (%).
- 6 / 24 is different from 6 % 24.
- This is different from: What is the remainder of 6 divided by 24?

Integer QUOTIENT

Operation

Integer REMAINDER

Operation

Expression Result vs Expression Result

- 14 / 4 3 vs 14 % 4 2 5 / 13 0 vs 5 % 13 5 7 / 2 3 vs 7 % 2 1 In simple words, if you are asked to give the quotient of an integer operation, use the usual division operator.
- Precedence of Arithmetic Operators: One of the critical things that needs to be considered when dealing with arithmetic operators is the precedence or the order in which operators will be evaluated first when they are combined in an expression.
- A common mnemonic to easily remember this rule is the PEMDAS or Please Excuse My Dear Aunt Sally, which technically represents Parentheses, Exponents, Multiplication, Division, Addition, and Subtraction.
- Another version of this rule is called BODMAS, which refers to Brackets (P = Parentheses), Order (E = Exponents), Division, Multiplication, Addition, and Subtraction.

3. Increment and Decrement Operators

- Increment and decrement, denoted by "++" and "--" respectively, are special types of unary operators (operators that require only one operand) that change the value of a variable before or after its value is used in an expression.

Symbol Operation Usage Result

- 6 Types of Operators in C and C++ | Enhance Your Fundamental Skills. .

Key terms

Multiplication and Division - at the same "priority level" as addition and Subtraction. In other words, if the

07. Types of Operators Part 2

Types of Operators Part 2

- Topic: Types of Operators Part 2

OPERATORS (continuation)

- However, programming is not always (or not only) a line-by-line (sequential) code processing.

4. Relational Operators

- As the name implies, these operators test or define the kind of relation between two or more entities.
- They are used to compare values stored in variables, to determine whether one value is equal to, less than, or greater than another.

Symbol Operator

Name

Usage Explanation

- == equal x == y Checks if the values of operands are equal or not; if yes, then the condition becomes true. != not equal x != y or x < > y

Checks if the values

- Note: Don't confuse the '=' (one equal sign) with '==' (two equal signs).

- In your Math subjects, you have been using the symbol '=' to mean equal and '≠' for not equal.
- However, in programming, the '≠' is not used and the '=' symbol is the assignment operator (review Topic 6).
- IF num1 == num2 PRINT "num1 and num2 are equal"; ELSE IF num1 > num2 PRINT "num1 is the bigger number"; ELSE //no other option left but num1 < num2 PRINT "num2 is the bigger number"; c.

ELSE

5. Logical Operators

- They are used to combine relational operators to make more complex decisions.
- Common logical operators include AND, OR, and NOT.
- AND and OR are also called Binary Logical Operators because they involve two variables, while NOT is a Unary Logical Operator because it only needs one operand/variable.

Symbol Operator

Name

Usage Explanation

- It is a table that shows the truth- value of one or more compound propositions for every possible combination of truth-values of the propositions making up the compound ones.

Logical

AND

Operator

TRUE TRUE TRUE

TRUE FALSE FALSE

FALSE TRUE FALSE

FALSE FALSE FALSE

Logical OR

Operator

TRUE TRUE TRUE

TRUE FALSE TRUE

FALSE TRUE TRUE

FALSE FALSE FALSE

Logical

NOT

Operator

TRUE FALSE

FALSE TRUE

- 25 < 7 || 15 > 36 False || False = False b.
- 14 > 7 && 5 <= 5 True && True = True c. ! (13 != 7) ! (True) = False d.

08. Flowcharting Part 1-Symbols and Usage

Flowcharting Part 1-Symbols and Usage

FLOWCHARTING

- Advantages of Flowcharts: 1.
- Effective Communication: Flowcharts are a clearer way of communicating the logic of the system compared to pseudocode.
- Easy Debugging and Efficient Testing: A program flowchart is a tool for programmers for the debugging and testing processes.
- Disadvantages of Flowcharts: 1.
- They include engineers, scientists, manufacturers, business managers, and health care workers.
- Below is a sample Order Process. -- 1 of 6 -- Here's another example of a high-level flowchart for an enrollment process: A Detailed Flowchart, on the other hand, provides a graphical representation of a process by showing all the steps and activities involved.
- This type of flowchart involves specific sequences, conditional points, and repetitive tasks.
- The following page provides an example of a detailed flowchart for an Order Process. -- 2 of 6 -- For programmers,

flowcharting is an effective way of outlining the flow of information and processes visually, which helps them to eventually come up with a coded solution.

- It is a practical tool for identifying variables, tracing errors, connecting code, and refining ideas.

1. Terminal / Terminator Block

2. Flow Line

- It is used to show the flow or direction of tasks in a process or program.

3. Preparation / Initialization / Declaration Block

Simple variable declaration

Declaration of variable with initial value

- It is used to declare (tell the computer of) and/or initialize variables needed for the program.

4. Input / Output Block

- Sample Input: Sample Output: It is used for getting inputs and displaying outputs.

5. Process Block

- It is used to indicate a process step (instructions or actions).

- This is the shape that contains statements for: a.

6. Decision / Condition Block

- It is used to represent decision-making statements.

7. Predefined Process Block

- It is used to call a specific function or subprogram (independent portions of a code or flowchart) - review Topic 4 about Subroutines.

8. On-page Connector

- It is used to connect blocks on the same page.

9. Off-page Connector

- It is used to connect blocks that are located on a different page.

10. Annotation

09. Flowcharting Part 2-General Rules and Tracing

Flowcharting Part 2-General Rules and Tracing

I. GUIDELINES FOR FLOWCHARTING

- A flowchart is an effective way of showing steps in a process or program but it can also get confusing and misleading if not constructed properly.

1. Follow Proper Direction

2. Use Correct Symbols

3. Use Flow Lines (Arrows) Not Lines

- All the blocks in a flowchart should be connected using flow lines.

- Using just lines (no arrow heads) produces confusion as it can mislead the reader from knowing which is the previous or next block. -- 1 of 5 --

4. Apply Proper Naming Conventions for Variables

- Here are some rules to follow: - Declare a variable with name and its data type before they are used. - A variable name is any valid identifier that is not a keyword or any reserved word in a programming language.

Case sensitive (num1 is different from Num1)

5. Ensure Entry and Exit Points on Symbols

6. Keep Everything on One Page

7. Use Line Jumps when Necessary

8. Trace Your Flowchart

II. TRACING

- Flowchart Tracing means simulating the execution of a program on paper (you may also use a document or spreadsheet) to scan through the diagram block-by-block, documenting the changes that are happening primarily on the variables.

- It is a technique that is used to test algorithms (flowchart or pseudocode) for logic errors.
- Here is a typical trace table.

Step Statement Notes num1 num2

Step Statement Notes a y

- 1 Input a User enters the value of 51 51 ?
- 2 $y = 5$ y is assigned the value of 5

Still 51 5

- 3 If $a * 2 > 100$ Evaluates whether this is true or false (since a is 51, multiplying it to 2 returns 102, it means that this condition is true; proceed to the YES part)

Still 51 Still 5

- 4 $a = a * 3$ a will take the value of 51 multiplied to 3 153 Still 5 $y = a * 4$ y will take the value of a (which is 153) multiplied to 4

Still

- 153 612 6 Output a, y The values 153, 612 are displayed.

Step Statement Notes num1 num2 Output

- 1 Input num1 User enters the value of 1 1 ?
- 2 $num2 = 11$ num2 is assigned the value of 11

Still 1 11

- 1st pass; returns YES Still 1 Still 11 4 Output num1 Still 1 Still 11 1 5 $num1 = num1 + 2$ num1 takes the value of 1 + 2
- 3 Still 11 6 $num1 \leq num2$?

YES

Still 3 Still 11

- 7 Output num1 Still 3 Still 11 1 3 8 $num1 = num1 + 2$ num1 takes the value of 3 + 2 5 Still 11 9 $num1 \leq num2$?

YES

Still 5 Still 11

- 10 Output num1 Still 5 Still 11 1 3 5 11 $num1 = num1 + 2$ num1 takes the value of 5 + 2 7 Still 11 12 $num1 \leq num2$?

YES

Still 7 Still 11

- -- 4 of 5 -- 13 Output num1 Still 7 Still 11 1 3 5 7 14 $num1 = num1 + 2$ num1 takes the value of 7 + 2 9 Still 11 15 $num1 \leq num2$?

YES

Still 9 Still 11

- 16 Output num1 Still 9 Still 11 1 3 5 7 9 17 $num1 = num1 + 2$ num1 takes the value of 9 + 2 11 Still 11 18 $num1 \leq num2$?

YES

Still 11 Still 11

- 19 Output num1 Still 11 Still 11 1 3 5 9 11 20 $num1 = num1 + 2$ num1 takes the value of 11 + 2 13 Still 11 21 $num1 \leq num2$?
- 7th pass; returns NO Still 13 Still 11 22 Program ends Bouras, A. (n.d.-b).

Key terms

Though you - not doing actual coding yet, it is good to already practice proper use of variable names. In Flowchart Tracing - simulating the execution of a program on paper (you may also use a document or

10. Initialization, End-User's View, and Input Validation

Initialization, End-User's View, and Input Validation

VARIABLE DECLARATION and INITIALIZATION

- You have previously learned that a variable is a storage/container.
- Variable Declaration basically means telling the computer that you need a variable or variables.
- Variable Initialization means assigning a starting value for a variable.

- In flowcharting, while the focus is on logic, it is a good practice to already incorporate some syntax rules.

1) **dataType variableName;**

2) **dataType variableName = initialValue;**

- The initial value must be of the correct data type.
- Examples: - int num1 = 10; - string greeting = "Hello!"; - float average = 3.5; int average = 3.5 will be wrong because 3.5 is a floating point (decimal) number so the data type must be a float.

3) **dataType variableNameOne, variableNameTwo;**

4) **dataType variableNameOne = initialValueOne, variableNameTwo = initialValueTwo;**

- The presence of garbage values must be avoided because these can cause some runtime issues during program execution.

END-USER'S VIEW

- Using the Swapping of 2 Integers flowchart that was previously presented in Lesson 5, you will notice that there is a big portion that an end-user cannot see. -- 2 of 6 -- In this case, as a programmer, you know that once this program runs, the computer will ask for 2 integer inputs from the end-user.
- Below is an example of a preview of the swapping program (assuming that the input values for num1 and num2 are 5 and 10 respectively).

Screen Preview

The new value for 1st Number is 10

The new value for 2nd Number is 5

INPUT VALIDATION

- Input validation, also known as data validation, is the testing of any input supplied by a user.
- It is a good practice to validate all input entered into a program.
- It is illogical to buy negative-numbered or zero pieces of something. - Booking a hotel and you are asked to select your check-in and check-out dates: The system should not allow you to select dates before the current date.
- If the input does not meet the condition, the program should either end the flow or allow the user to input again.
- There must be validations for every input.
- Hence, on the next column is a far better flowchart. -- 5 of 6 -- Syntax of Variable Declaration. (n.d.).

11. Basic Calculator Algorithm

Basic Calculator Algorithm

CALCULATOR ALGORITHM

Screen Preview

This is a Simple Calculator Program

- To create an algorithm for this program (and any other program), it is helpful to ask the following algorithmic questions: - What are the needed variables?
- You might initially think that there should be variables to store the sum, difference, product, or quotient.
- There will be no problem if you choose that approach, however, having one variable to store whatever result is already sufficient. - What are the data types and initial values of these variables?
- Going back to the program description, it should accept any kind of number (positive, negative, zero, decimal) so this means that in declaring the variables to store the numbers, it should be a safe data type.
- An int (integer) data type will not be able to accept decimal numbers but a float can accept even whole numbers so it is the best data type to use.
- For the operator, a char (character) data type is to be used.
- The variable declarations would be: - float num1=0, num2=0, result=0 - char operator='?' - What are the inputs?
- Other than the expected outputs, the program should be displaying some instructions for the user.
- There should be prompts within the program that tells the user what he/she should do.
- Bye!" Note: When displaying words on the screen, they should be enclosed in quotation marks but when displaying the values of variables, just put the variable name without quotation marks. - What are the processes?
- If there is any use of the Assignment, Arithmetic, Increment, and Decrement Operators within the program, it means that there are processes to be performed.
- Remember that programming is a creative process.

12. The Real Number System

The Real Number System

I. REAL NUMBER

- If you want to master the latter, you should also have a good foundation, or at least interest, in the former.
- These are the same things Mathematics teaches us.
- Image 1: The Number Line There are different types of real numbers and these include Natural numbers, Whole numbers, Integers, Fractions, Rational numbers, and Irrational numbers.
- Here is how they are categorized: -- 1 of 6 -- Image 2: Real Number System Diagram Here is another way of classifying real numbers with some examples: Image 3: Real Number Classification with Examples In programming, you will be dealing with all these kinds of numbers.
- Other than the main types of real numbers above, they can also be classified according to their properties and representation.

II. POSITIVE and NEGATIVE NUMBERS

- You can translate these criteria into a programming language which means that a number is positive if $\text{num} > 0$ and it is negative if $\text{num} < 0$.
- Once zero is inputted, the program shall ask for another integer input because it is a neutral number.

Screen Preview

- Enter a Number: 0 Zero is a neutral number.

III. EVEN and ODD NUMBERS

- Examples: - $10 \% 2 = 0$; therefore 10 is an even number -- $4 \% 2 = 0$ -- $9 \% 2 = 1$; therefore 9 is an odd number Let us create a flowchart that will ask an integer input from the user and it will tell whether the number is even or odd.
- While zero is considered a neutral number for the positive-negative program, it is considered even by principles of Mathematics because it passes the definitions and tests for an even number.
- This means adding a variable that will store the choice of the user to repeat the program or not.
- It is another variable that is used to keep track of anything that needs to be counted.

Screen Preview for Improved Positive-Negative

Screen Preview for Improved Even-Odd

- Retrieved July 30, 2021, from <https://www.britannica.com/story/is-zero-an-even-or-an-odd-number> Real Number System - Other types of real numbers. (n.d.).

Key terms

Mathematics and Computer Programming - intertwined. If you want to master the latter, you should also

Positive numbers - integers that are greater than zero (right side of zero in the number line) while

Even numbers - integers that are divisible by 2. They end with digits 0, 2, 4, 6, or 8 while odd numbers are

13. Multiple and Nested Conditions

Multiple and Nested Conditions

I. MULTIPLE and NESTED CONDITIONS

- As you have learned in Lesson 5, programs are not always sequential (simply performing codes line-by-line).
- When there are multiple possible paths, nested or chained conditions should be applied.
- Some Applications: Video games are a great example of the use of conditional statements.

Player is dead or disabled

- It is a simple Artificial Intelligence algorithm that uses a set of conditional statements (yes or no questions) to pinpoint the answer.

II. CONDITION-DEPENDENT OUTPUTS

Grade Range Remarks

Exactly 75 Passed

- 76 - 80 Needs Improvement 81 - 85 Satisfactory 86 - 90 Good 91 - 95 Very Good 96 - 100 Excellent Given the description, you need to prepare a variable that will store the grade input.
- Since a grade can have decimal values, float is the most appropriate data type to be used.

- The program shall also allow the user to input -- 2 of 5 -- another grade if he/she wants therefore another variable of char data type shall be declared that will store the choice input of the user.

Key terms

- Video games - great example of the use of conditional statements. In programming, a conditional
- Minecraft - another game which is a perfect example of a program that uses a multiple condition structure

14. Number Sequences

Number Sequences

I. NUMBER SEQUENCE DEFINED

- A number series/sequence is a progression or list of numbers that are linked by a certain logical rule or numerical pattern.
- Numbers in a sequence are called terms.

Find the next number in the sequence

- In the example above, there is a difference of 6 between each number, therefore, the rule for this sequence is to add 6 to get the next number.
- It is an essential tool, among other tests, used by companies to assess someone's intelligence.

II. COMMON TYPES OF NUMBER SEQUENCES

1. Arithmetic Sequence

2. Geometric Sequence

3. Incremental Sequence

4. Triangular Number Sequence

5. Square Sequence

- $1 = 1 * 1$ $4 = 2 * 2$ $9 = 3 * 3$ $16 = 4 * 4$ $25 = 5 * 5$ etc...

6. Cube Sequence

- $1 = 1 * 1 * 1$ $8 = 2 * 2 * 2$ $27 = 3 * 3 * 3$ $64 = 4 * 4 * 4$ $125 = 5 * 5 * 5$ etc...

7. Fibonacci Sequence

- It is a unique sequence that was said to have been discovered or "invented" by Leonardo Fibonacci, an Italian mathematician who was born around A.D.
- 1170 and was also known as Leonardo of Pisa.
- It is a -- 2 of 6 -- sequence of numbers in which the first two numbers are 0 and 1 and each succeeding number is the sum of the previous two.
- $2 =$ adding the two numbers before it $(1+1)$ $3 =$ adding the two numbers before it $(1+2)$ $5 =$ adding the two numbers before it $(2+3)$ etc...

III. FLOWCHARTING NUMBER SEQUENCES

- From-to means that the start and end values are included in the instruction or display while between means that the start and end values are NOT included in the instruction or display.
- If there is a programming instruction that tells you to display integers from 5 to 10, 5 and 10 will be displayed along with 6, 7, 8, and 9.
- 1 2 3 4 5 6 7 8 9 10 -- 3 of 6 -- Example #2: Create a flowchart that will display all even numbers from 2 to 30.
- 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 -- 4 of 6 -- Example #3: Create a flowchart that will display all odd numbers between 1 and 25.
- 3 5 7 9 11 13 15 17 19 21 23 You may want to practice improving these 3 examples by doing the following: - Adding a loop, allowing the user to repeat the program just like what you have been doing in the previous lessons and exercises.
- Creating a program that combines these 3.
- Ask the user to select what program he/she would like to run (Even Series, Odd Series, etc.) - Asking an input from the user regarding how many numbers he/she wants to display rather than forcing the terminating value. - Displaying the process of computing the next numbers. - Creating a program that combines all the...
- 101 Computing. <https://www.101computing.net/number-sequences/> Number Sequences - Square, Cube and Fibonacci. (n.d.).
- Retrieved August 3, 2021, from <https://www.mathsisfun.com/numberpatterns.html> What is a number sequence? .

15. Design Patterns in Programming

Design Patterns in Programming

I. "STAR" PATTERNS/DESIGNS

II. FLOWCHARTING DESIGNS

- Sequentially-forced Approach It is the way of displaying each part of the design line-by-line, avoiding any instance of conditional and repetitive structure.
- This is the easiest way to produce the desired output, however, it does not apply the principle of algorithmic mindset.
- It is also a "boxed solution" which means that it only works for a particular situation.
- For example, the program is expected to produce a right triangle occupying 5 rows and 5 columns like this: * -- 3 of 8
-- ***** A forced approach means displaying each line of the design sequentially whether in flowchart or actual coding.
- In real life, programs should be flexible and are able to produce the expected results based on the input and other interactions of the user with the program.
- Condition-and-Loop-Driven Approach It is the way of displaying the design by carefully studying the logic of displaying each asterisk (or whatever special character you want) which builds up until the whole design is completed.
- It refers to the effective use of conditional and repetitive statements in order to produce the desired result.
- It definitely looks more complicated than the forced approach but it is the best application of algorithmic thinking.
- If the user inputs 5 then 5 rows and 5 columns in the grid will be occupied) but you should not force 5 as a terminating number in the decision box because the user should be able to decide on what he/she wants.
- Instead, use this variable to store whatever number the user decides to use. - row This is the variable that points to the current position of the row. - col This is the variable that points to the current position of the column.
- The program should repetitively display an asterisk until it reaches the numth row and the numth column.

Screen Preview

Screen Preview

- Exploring the UI Universe: Different Types of UI.

Key terms

- CLI - one the common interfaces (how a user interacts with the computer) used to see the outputs of

16. Measures of Central Tendency

Measures of Central Tendency

I. MEASURES OF CENTRAL TENDENCY DEFINED

- The "measures of central tendency" (also known as measures of center or central location) is a key concept in Statistics.
- You would always hear the word "average" in scores or points in a particular group because it describes what is typical, normal, usual, or representative.
- The three most common measures of central tendency are the mean, median, and mode.

1. Mean

- It is the sum of all values divided by the total number of values.
- Practical applications: - Used in schools to get students' average marks - Used to calculate the mean salary of individuals in a certain field so that they can know what type of "average" salary to offer to new employees (for Human Resources) - Used in effective budgeting by computing a family's average income and e...

2. Median

- It is the middle value in an ordered/sorted list of numbers.
- It is the measure that divides the population into the top and bottom 50%.
- If the data set has an odd number of data values, then the middlemost value of the sorted data set is the median.
- You can easily identify it from the list but for complex lists, you may use the computation $(\text{num}+1)/2$ where num is the number of elements in the data set.
- If the data set has an even number of data values, the average of two middle values is the median.

Therefore, median is 3

Mean of 3rd and 4th numbers

3. Mode

- It is the most common number in the set of data or the value that occurs most often.
- The data value that occurs most is the mode of the dataset.

Range

- It is a measure of dispersion/spread and not of central tendency but it is often associated with computations involving mean, median, and mode.
- Range is the difference between the smallest and largest values.

A Helpful Mnemonic

Hey diddle diddle,

The Mode is the one that appears the most,

- and the Range is the difference between.

II. FLOWCHARTING THE ARITHMETIC MEAN

Screen Preview

- 1) Press 1 then M+ (now contains 1)
- 2) Press 2 then M+ (now contains 3)
- 3) Press 3 then M+ (now contains 6)
- 4) Press 4 then M+ (now contains 10)
- 5) Press 5 then M+ (now contains 15)
- 6) Press MR (displays the latest value of the memory which is 15)
- 7) Press / 5 (divide by 5)
- 8) You get an average of 3
- 9) Press MC to reset the memory for future use

- Here are the purposes of the variables used: - num - temporarily stored every valid input - numplus - kept on adding every valid number entered; it functioned like the calculator's M+ - mean - stored the mean of all the numbers entered - count - counted the number of values entered Allen, M. .
- Retrieved August 5, 2021, from <https://www.abs.gov.au/websitedbs/D3310114.nsf/Home/Statistical+Language+-+measures+of+central+tendency+What+is+the+Importance+of+Measures+of+Central+Tendency+in+Statistics?> .

17. Introduction to Arrays

Introduction to Arrays

I. DEFINITION OF ARRAYS

- An array is a collection of elements of the same data type that are stored in adjacent memory locations, where each of these elements can be accessed through its index.
- A structured data type means that each data item is a collection of other data items.
- The purpose of structured data types is to group related data for convenient access using the same identifier.

1. integral (char, short, int, long, and bool)

2. enum

3. floating (float, double, long double)

1. array

2. struct

3. union

4. class

- There are two types of arrays.
- An Excel spreadsheet is an example of a multidimensional array.

John Mark Paul David Daniel

- -- 1 of 7 -- Multidimensional: A table of grades - aname[3][5] 0 1 2 3 4 0 95 98 97 85 76 1 88 89 90 78 80 2 92 94 90 88 87

II. REAL-WORLD APPLICATIONS OF ARRAYS

- Its drawers are the indices (plural of index) and the things inside each drawer (index) are the elements (values).
- But in an array, elements should be of the same type.

Egg Box

- This is an example of a 2D array. -- 2 of 7 --

Mobile Phone Contacts

- They are stored in the form of an array.

ECommerce Sites

- Online stores such as Amazon, Ebay, Lazada, and Shopee are using tabular storage and presentation of their products which is an application of arrays.

III. USING ARRAYS IN PROGRAMMING

Declaring Arrays

- Just like any other variable, arrays should be declared before they can be used in the program.

Initializing Arrays

- Initializing means putting initial values to an array.
- For example, num[10] is an array with 10 elements/compartments where the first element is located at index 0 and the last is located at index 9.

Accessing Array Elements

- Examples: - num[3] = 5 means you are storing the value of 5 to the 4th element (counting of index starts at 0). - num[7] = -7 means you are storing the value of -7 to the 8th element. - num[2] = num[0] + num[1] means you are storing the sum of the values of the first and second compartments to the third element.
- Examples: - num[4] > 0? means that you are verifying if the value inside the 5th compartment of the array is greater than zero. - num[3] <= num[2]? means that you are comparing the values that are stored in the 4th and 3rd compartments of the array. - num[0] %2 == 0? means that you are testing if the value that is s...

Array Index as a Counter

IV. FLOWCHARTING ARRAYS

Screen Preview

- Without using an array, the program on the previous page would require 50 variables for the names but with array implementation, you just need to declare 1 array variable of string data type that has a size of 50 and 1 variable for the index that goes from one element of the array to another.

Screen Preview

- 5 Different Application of array in real life.

Key terms

- Initializing - putting initial values to an array. For example, num[10] is an array with 10

18. Overview of C++ Programming

I. BACKGROUND

- C++ is a general-purpose programming language that was developed as an enhancement of or successor to the C language.
- Here are examples of a C++ source code and the screen preview: -- 1 of 7 -- C++ is one of the frequently-used high-level languages.
- Others include C#, Java, and Visual Basic.
- A source code is the original (raw) program in a high-level language while an object (machine) code is the translated version of the former into a language that the computer can understand and execute.
- Installable softwares used to compile a C++ high level source code include Microsoft Visual Studio, Turbo C++ and Dev C++.

II. C++ BASICS

- Note that they are ignored by the compiler and should therefore not cause any syntax errors (as long as they are properly used).
- There are two types of comments:

1. Single Line Comment //this is a comment

2. Multi-line/Block Comment /*this is a

- Here is the starting C++ compile environment of Online GDB with comments in green: -- 2 of 7 -- b.

- Preprocessor Directives `#include <iostream>` This tells the processor to include the input/output stream header file. `iostream` is a header library that allows you to use input and output objects such as "cin" (the keyword for getting inputs) and "cout" (the code for displaying something on the screen).
- There are several libraries in C++ which contain information that are necessary and useful in creating programs but `iostream` is a default one that needs to be included. using namespace std; This statement allows you to use cin and cout without the prefix std::.
- It means that if you do not include this statement, then cout should be used as std::cout and endl should be used as std::endl. c.
- White Spaces These refer to blank lines, space characters, and tabs.
- The "main" Function Exactly one function in a program must be the "main".
- It is considered as a keyword or a reserved word in C++, therefore, it should not be used to name a variable.
- If in flowcharting the Start inside a terminal block is where the execution of the program begins, in C++ it is the main function.
- Here is an example of a C++ source code that displays the words "Hello World!" on the screen:

```

/*****This is a C++ program that displays "Hello World!" *****/
#include <iostream> using namespace std; int main() { cout<<"Hello World"; return 0; } Go to www....

```
- Click the Run button and at the bottom part you should see the output. e.
- Types of statements include: - return statement - returns the value of a variable after going through some processes and also terminates the program - cout - (pronounced "see-out") is used together with the insertion operator (<<) to output text.
- This is equivalent to the Output block in flowcharting. - cin - (pronounced "see-in") is used together with the extraction operator (>>) to get input from the user.

III. SOME SIMPLE C++ PROGRAMS

Screen Preview

- Enter Name: Joseph Enter Age: 17 Enter Email Address: joseph@domain.com Here is the information you entered: You are Joseph, 17 years old, with an email address of joseph@domain.com.

Screen Preview

- C++ Source Code

```
#include <iostream> using namespace std; int main() { int num; //ask for user input cout<<"Enter Number: "; cin>>num; /*compare if positive or negative*/ if(num>0) cout<<"It is Positive!"; else //means num<0 cout<<"It is Negative!"; return 0; }
```

 Actual Output in OnlineGDB: Copy and paste the codes sep...